

UNIVERSIDADE FEDERAL DE LAVRAS

ESPECIFICAÇÃO DE REQUISITOS DE SOFTWARE: API REST e interface web para controle de contratos, pagamentos, cooperandos e despesas de uma cooperativa de transportes

2026

UNIVERSIDADE FEDERAL DE LAVRAS

Projeto de disciplina: API REST e interface web para cooperativa de transportes

ESPECIFICAÇÃO DE REQUISITOS DE SOFTWARE: API REST e interface web para controle de contratos, pagamentos, cooperandos e despesas de uma cooperativa de transportes

Arthur Valadares Campideli
Discente

Júlia Ribeiro Souza
Discente

Lavras
2026

RESUMO

Este relatório formaliza os requisitos de um sistema composto por uma API REST e uma interface web para apoiar a gestão de uma cooperativa de transportes, cujo contrato de prestação de serviço é firmado entre cliente e cooperando, cabendo à cooperativa intermediar a cobrança, reter uma taxa configurável por contrato e repassar o valor líquido ao cooperando. O levantamento foi conduzido por entrevistas estruturadas com perguntas de múltipla escolha, em três rodadas, e resultou em regras de negócio, requisitos funcionais e não funcionais, casos de uso, modelo de dados com seis tabelas, contrato de recursos da API, requisitos da interface web, matriz de permissões por perfil (administrador, financeiro, motorista e cliente) e critérios de aceitação. O desenvolvimento adota o modelo em cascata, com controle de versão segundo o Git Flow. O documento serve de referência única para projeto, implementação e testes, atendendo às exigências acadêmicas de um CRUD de uma tabela e de CRUDs que relacionam ao menos três tabelas.

Palavras-chave: Engenharia de requisitos; API REST; Interface web; Cooperativa de transportes; Contratos; Git Flow.

LISTA DE ILUSTRAÇÕES

Figura 1 – Modelo em cascata; setas tracejadas indicam realimentação para a fase anterior. . . . 3
Figura 2 – Git Flow aplicado ao projeto: ramos, ciclo de *release* e *hotfix* (setas = criação de ramo ou *merge*). . . . 4
Figura 3 – Ciclo de engenharia de requisitos aplicado. . . . 6
Figura 4 – Diagrama de contexto. . . . 9
Figura 5 – Casos de uso do módulo de acesso e cadastros. . . . 9
Figura 6 – Casos de uso do módulo de contratos. O administrador também atua em UC06 e UC07. . . . 10
Figura 7 – Casos de uso do módulo financeiro. O motorista só enxerga e lança dados próprios (RN10). . . . 10
Figura 8 – Arquitetura em camadas e fluxo de uma requisição. Aspectos transversais: auditoria (`criado_em`, usuário), *logs* e configuração por ambiente. . . . 19
Figura 9 – Diagrama entidade-relacionamento (PK = chave primária, FK = estrangeira, UK = única). Veículos não são modelados. . . . 20
Figura 10 – Máquina de estados do contrato (RN12). Aprovar gera os pagamentos na mesma transação. ENCERRADO e CANCELADO são finais. . . . 23
Figura 11 – Máquina de estados do pagamento (RN04, RN06, RN07). *Em atraso* é derivado: situação PREVISTO com vencimento anterior à data atual (P5). . . . 23
Figura 12 – Fluxo do dinheiro: o contrato é entre cliente e cooperando; a cooperativa intermedeia. . . . 23
Figura 13 – Sequência: da elaboração do contrato ao repasse (as chamadas partem da interface web e cada passo valida papel, escopo e estado). . . . 24
Figura 14 – Mapa de navegação por papel (estrutura de telas, sem projeto visual). As despesas e os contratos em rascunho também usam o formulário. . . . 25

LISTA DE QUADROS

Quadro 1 Glossário do domínio. 2
Quadro 2 Fases, artefatos e relação com o controle de versão. 3
Quadro 3 Convenções de ramificação. 4
Quadro 4 Partes interessadas e atores. 6
Quadro 5 Decisões registradas nas rodadas de elicitação. 7
Quadro 6 Regras de negócio. 11
Quadro 7 Requisitos não funcionais, com categorias de qualidade conforme a ISO/IEC 25010 (ISO/IEC, 2011). 16
Quadro 8 Mapeamento entre a restrição da disciplina e a especificação. 18
Quadro 9 Dicionário de dados: entidade `usuario` 20
Quadro 10 Dicionário de dados: entidade `cooperando` 20
Quadro 11 Dicionário de dados: entidade `cliente` 21
Quadro 12 Dicionário de dados: entidade `contrato` 21
Quadro 13 Dicionário de dados: entidade `pagamento` 21
Quadro 14 Dicionário de dados: entidade `despesa` 22
Quadro 15 Recursos da API. O prefixo `RF-` foi omitido na última coluna. 26
Quadro 16 Autorização por papel (RBAC) com restrição por propriedade do recurso. 28
Quadro 17 Cenários de aceitação. 29
Quadro 18 Registro de riscos. 30

LISTA DE ABREVIATURAS E SIGLAS

ABNT Associação Brasileira de Normas Técnicas
API Application Programming Interface
CORS Cross-Origin Resource Sharing
CRUD Create, Read, Update, Delete
DER Diagrama Entidade-Relacionamento
HTTP Hypertext Transfer Protocol
JSON JavaScript Object Notation
RBAC Role-Based Access Control
REST Representational State Transfer
RF Requisito Funcional
RN Regra de Negócio
RNF Requisito Não Funcional
SPA Single-Page Application
SQL Structured Query Language
UC Caso de Uso (Use Case)

SUMÁRIO

1 INTRODUÇÃO 1
1.1 PROPÓSITO E ESCOPO 1
1.2 MINIMUNDO 1
1.3 FORA DE ESCOPO 1
1.4 GLOSSÁRIO 2

2 PROCESSO DE DESENVOLVIMENTO 3
2.1 MODELO EM CASCATA 3
2.2 FLUXO DE RAMIFICAÇÃO (GIT FLOW) 4

3 LEVANTAMENTO DE REQUISITOS 6
3.1 TÉCNICA E CICLO 6
3.2 PARTES INTERESSADAS 6
3.3 RASTREABILIDADE DAS DECISÕES 7

4 VISÃO GERAL E CASOS DE USO 9
4.1 CONTEXTO DO SISTEMA 9
4.2 CASOS DE USO 9

5 REGRAS DE NEGÓCIO 11

6 REQUISITOS FUNCIONAIS 12
6.1 AUTENTICAÇÃO E ACESSO 12
6.2 USUÁRIOS — *CRUD DE UMA TABELA* 12
6.3 COOPERANDOS E CLIENTES 12
6.4 CONTRATOS — *CRUD RELACIONADO (CONTRATO, COOPERANDO, CLIENTE, PAGAMENTO)* 13
6.5 PAGAMENTOS E REPASSES 13
6.6 DESPESAS — *CRUD RELACIONADO (DESPESA, COOPERANDO, CONTRATO, USUÁRIO)* 14
6.7 RELATÓRIOS 14
6.8 INTERFACE WEB 14

7 REQUISITOS NÃO FUNCIONAIS 16

8 RESTRIÇÕES E ATENDIMENTO AO ESCOPO ACADÊMICO 18

9 MODELAGEM E PROJETO PRELIMINAR 19
9.1 ARQUITETURA EM CAMADAS 19
9.2 MODELO DE DADOS 19
9.3 ESTADOS DO CONTRATO E DO PAGAMENTO 23
9.4 FLUXO FINANCEIRO 23
9.5 SEQUÊNCIA DO CICLO DE VIDA DE UM CONTRATO 24
9.6 MAPA DE NAVEGAÇÃO DA INTERFACE WEB 25

10 CONTRATO DA API (RECURSOS E OPERAÇÕES) 26

10.1 CONVENÇÕES.....26
11 MATRIZ DE PERMISSÕES.....28
12 CRITÉRIOS DE ACEITAÇÃO.....29
13 RISCOS E GESTÃO DE MUDANÇAS.....30
REFERÊNCIAS.....31

1 INTRODUÇÃO

1.1 PROPÓSITO E ESCOPO

Este documento formaliza os requisitos do sistema de apoio à gestão de uma cooperativa de transportes, composto por uma **API REST** e por uma **interface web** que a consome, desenvolvido como projeto da disciplina de banco de dados/desenvolvimento de APIs. Ele é a referência única para projeto, implementação e testes (modelo em cascata, Capítulo 2) e é o artefato de saída da fase de levantamento. A organização dos requisitos segue as recomendações de engenharia de requisitos da norma ISO/IEC/IEEE 29148 (ISO/IEC/IEEE, 2018).

O sistema cobre: cadastro e controle de acesso de usuários; cadastro de cooperandos e clientes; contratos entre cliente e cooperando com ciclo de aprovação; pagamentos programados com baixa e repasse manuais; despesas da cooperativa e dos cooperandos; e consultas/relatórios financeiros. A interface web oferece a cada perfil as telas de acesso a essas funções.

1.2 MINIMUNDO

A cooperativa reúne motoristas autônomos (**cooperandos**) que prestam serviços de transporte a **clientes**. O contrato de prestação é firmado **entre o cliente e o cooperando**; a cooperativa não é parte do contrato, mas intermedeia o fluxo financeiro: recebe o pagamento do cliente, retém a **taxa da cooperativa** e repassa o **valor líquido** ao cooperando. A taxa é definida por contrato, em percentual ou em valor fixo.

A cooperativa **não administra os veículos** dos cooperandos; portanto, veículos não são modelados. Quatro perfis interagem com o sistema: administrador da cooperativa, financeiro, motorista (cooperando) e cliente (somente consulta).

1.3 FORA DE ESCOPO

- Cadastro e gestão de veículos, manutenção ou rotas.
- Integração com meios de pagamento, bancos ou emissão fiscal: toda baixa e todo repasse são registrados manualmente.
- Aplicativo móvel nativo e projeto visual (identidade visual, protótipos de alta fidelidade); a interface web é especificada por requisitos funcionais e de qualidade, não por padrão estético.
- Abatimento automático de despesas do cooperando no repasse (despesas são informativas, ver RN09).

1.4 GLOSSÁRIO

Quadro 1 – Glossário do domínio.

Termo	**Definição**
Cooperativa	Entidade que intermedeia cobrança e coleta de taxa; é o “dono” do sistema.
Cooperando	Motorista associado; prestador do serviço e parte do contrato.
Cliente	Contratante do serviço de transporte; parte do contrato.
Contrato	Acordo entre um cliente e um cooperando, com valor, modalidade de pagamento e taxa.
Taxa	Valor retido pela cooperativa; percentual sobre o pagamento ou valor fixo por pagamento.
Baixa	Registro manual de que o cliente pagou uma parcela à cooperativa.
Repasse	Registro manual de que a cooperativa transferiu o valor líquido ao cooperando.
Valor bruto / líquido	Bruto: valor cobrado do cliente. Líquido: bruto menos a taxa.
RBAC	*Role-Based Access Control*: autorização por papel do usuário.
Interface web	Aplicação executada no navegador (SPA) que consome a API e apresenta as funções a cada perfil.

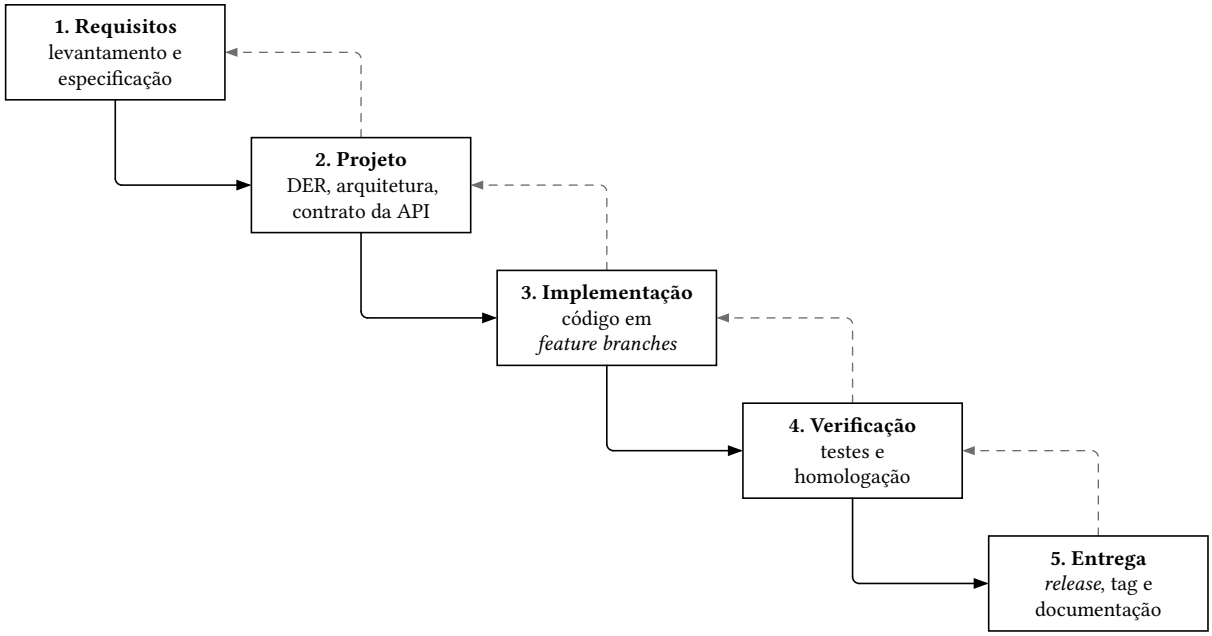
Fonte: Elaborado pelos autores.

2 PROCESSO DE DESENVOLVIMENTO

2.1 MODELO EM CASCATA

Adota-se o modelo **cascata** (*waterfall*) com fases bem definidas e marcos de aprovação ao fim de cada uma. Como o escopo é pequeno, estável e acadêmico, as fases são sequenciais; admite-se retorno controlado à fase anterior quando uma inconsistência é detectada (realimentação de Royce) (ROYCE, 1970; PRESSMAN; MAXIM, 2016), sempre registrando a mudança neste documento.

Figura 1 – Modelo em cascata; setas tracejadas indicam realimentação para a fase anterior.



Fonte: Elaborado pelos autores.

Quadro 2 – Fases, artefatos e relação com o controle de versão.

Fase	Artefatos de saída	Onde vive no Git	Critério de saída (marco)
1. Requisitos	Este documento; glossário; regras de negócio; casos de uso.	`docs/` em `develop` (PR revisado).	Documento aprovado pelo stakeholder.
2. Projeto	DER e dicionário de dados; arquitetura em camadas; esboço OpenAPI; matriz de permissões.	`docs/` e `openapi.yaml` em `develop`.	Modelo e contrato da API revisados.
3. Implementação	Código-fonte da API e da interface web, *migrations*, testes unitários.	`feature/*` mesclado em `develop`.	Todos os RF essenciais implementados.

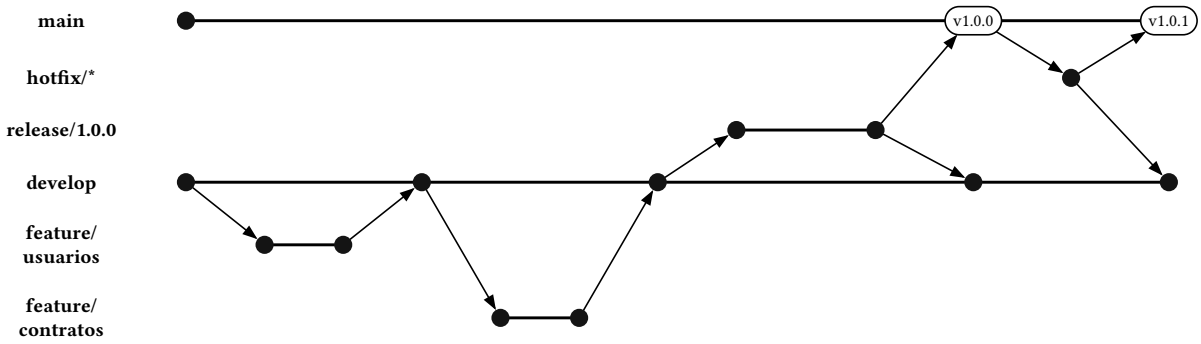
Fase	Artefatos de saída	Onde vive no Git	Critério de saída (marco)
4. Verificação	Testes de integração por endpoint; correções.	`release/x.y.z` a partir de `develop`.	Critérios de aceitação cumpridos.
5. Entrega	Versão estável, *tag* semântica, README e coleção de requisições.	Merge em `main` + tag `vX.Y.Z`.	Entrega e demonstração.

Fonte: Elaborado pelos autores.

2.2 FLUXO DE RAMIFICAÇÃO (GIT FLOW)

O Git Flow (DRIESSEN, 2010) organiza o trabalho em ramos de vida longa (`main`, `develop`) e de vida curta (`feature/*`, `release/*`, `hotfix/*`). Cada ramo `feature` implementa um ou mais requisitos rastreáveis (ID do RF no nome do ramo).

Figura 2 – Git Flow aplicado ao projeto: ramos, ciclo de *release* e *hotfix* (setas = criação de ramo ou *merge*).



Fonte: Elaborado pelos autores.

Quadro 3 – Convenções de ramificação.

Ramo	Origem → Destino	Nome	Regras
`feature`	`develop` ⟶ `develop`	`feature/RF-CTR-04`	Um ramo por requisito (ou grupo coeso). Merge via *pull request* com `--no-ff` e revisão de outro membro.
`release`	`develop` ⟶ `main` e `develop`	`release/1.0.0`	Somente correções e documentação. Congela o escopo; ao fim, merge em `main` com tag.
`hotfix`	`main` ⟶ `main` e `develop`	`hotfix/1.0.1`	Defeito crítico em versão entregue; incrementa o *patch*.
`main`	—	—	Sempre estável e etiquetado por versionamento semântico (PRESTON-WERNER, 2013). Sem *commits* diretos.

Ramo	Origem → Destino	Nome	Regras
`develop`	—	—	Integração contínua do trabalho; sempre compila e passa nos testes.

Fonte: Elaborado pelos autores.

Mensagens de *commit* seguem o padrão *Conventional Commits* (CONVENTIONAL COMMITS, [s. d.]): `tipo(escopo): descrição (ID-do-requisito)`, por exemplo `feat(contrato): aprovar contrato e gerar pagamentos (RF-CTR-04)`. Tipos: `feat`, `fix`, `docs`, `test`, `refactor`, `chore`. O código da API e o da interface ficam no mesmo repositório (pastas `api/` e `web/`), e o escopo do *commit* indica a parte alterada (por exemplo `api` ou `web`).

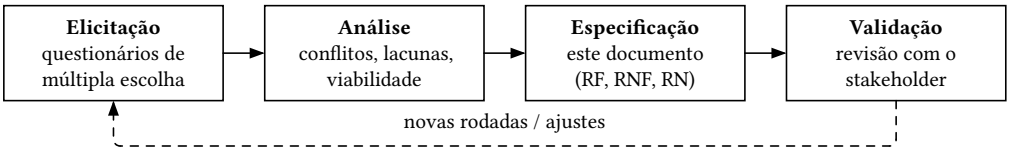
Definição de pronto de um requisito: implementado, com teste automatizado das regras de negócio associadas, documentado no OpenAPI e mesclado em `develop` por *pull request* aprovado.

3 LEVANTAMENTO DE REQUISITOS

3.1 TÉCNICA E CICLO

A elicitação foi feita por **entrevista estruturada com perguntas fechadas de múltipla escolha**, aplicadas em rodadas. A técnica é adequada ao porte do projeto, pois reduz ambiguidade, acelera a decisão e produz registro direto das escolhas (SOMMERVILLE, 2018; WIEGERS; BEATTY, 2013). O ciclo clássico de engenharia de requisitos (elicitação, análise, especificação e validação) foi percorrido em três rodadas: (1) núcleo do modelo, atores e taxa; (2) fluxo do dinheiro, despesas e ciclo do contrato; (3) cliente, pagamento/repasse e processo.

Figura 3 – Ciclo de engenharia de requisitos aplicado.



Fonte: Elaborado pelos autores.

3.2 PARTES INTERESSADAS

Quadro 4 – Partes interessadas e atores.

Papel	Interesse	Capacidade no sistema
Administrador	Governança: cadastros, aprovação de contratos, visão geral.	Gerencia usuários, cooperandos e clientes; aprova, encerra e cancela contratos; vê tudo.
Financeiro	Cobrança, baixa, repasse, despesas e saldo.	Registra baixas e repasses; lança despesas da cooperativa; emite relatórios.
Motorista (cooperando)	Seus contratos, o que receber e suas despesas.	Elabora e submete contratos; registra suas despesas; consulta apenas seus dados.
Cliente	Acompanhar o que contratou e pagou.	Consulta (somente leitura) seus contratos e pagamentos.
Equipe de desenvolvimento / docente	Atender à restrição acadêmica e avaliar.	Não é ator do sistema.

Fonte: Elaborado pelos autores.

3.3 RASTREABILIDADE DAS DECISÕES

Quadro 5 – Decisões registradas nas rodadas de elicitação.

Rod.	Pergunta	Decisão do stakeholder	Efeito na especificação
1	Tabelas do CRUD principal.	Cooperando, Contrato, Pagamento, Despesa e controle de usuário/ acesso.	Seis tabelas no DER (inclui Cliente, por decisão da rodada 3).
1	Perfis de usuário.	Administrador, Motorista, Financeiro e Cliente (consulta).	Quatro papéis RBAC; matriz de permissões (Capítulo 11).
1	Cobrança da taxa.	Configurável: percentual ou fixo, por contrato.	Campos `taxa_tipo` e `taxa_valor` no contrato; RN02 e RN03.
2	Fluxo do dinheiro.	Cliente paga à cooperativa, que repassa o líquido ao cooperando.	Pagamento com baixa e repasse; RN04 e RN07.
2	Origem das despesas.	Ambas, distinguidas por tipo/origem.	Campo `origem` em Despesa; RN08.
2	Ciclo do contrato.	Rascunho ⟶ pendente de aprovação ⟶ ativo ⟶ encerrado/cancelado.	Máquina de estados do contrato; RN12.
3	Representação do cliente.	Tabela própria (nome, documento, contato) vinculada a um usuário e aos contratos.	Entidade Cliente 1:1 com Usuário; RN14.
3	Pagamentos e repasses.	Pagamento único ou mensal programado; baixa manual; sem integração de pagamento.	Modalidade ÚNICO/ MENSAL; parcelas geradas na aprovação; RN05.
3	Processo de desenvolvimento.	Cascata com fases bem definidas e Git Flow.	Capítulo 2.
4	Interface web.	Necessária; faz parte do escopo do projeto.	Requisitos RF-WEB-01 a 11 e RNF-16 a 21; premissas P6 a P8.

Fonte: Elaborado pelos autores.

Premissas adotadas pela equipe (a validar com o stakeholder; ver também a seção de riscos): (P1) cooperandos e clientes são cadastrados pelo administrador juntamente com seu usuário de acesso; (P2) taxa fixa incide por pagamento (parcela); (P3) despesas do cooperando

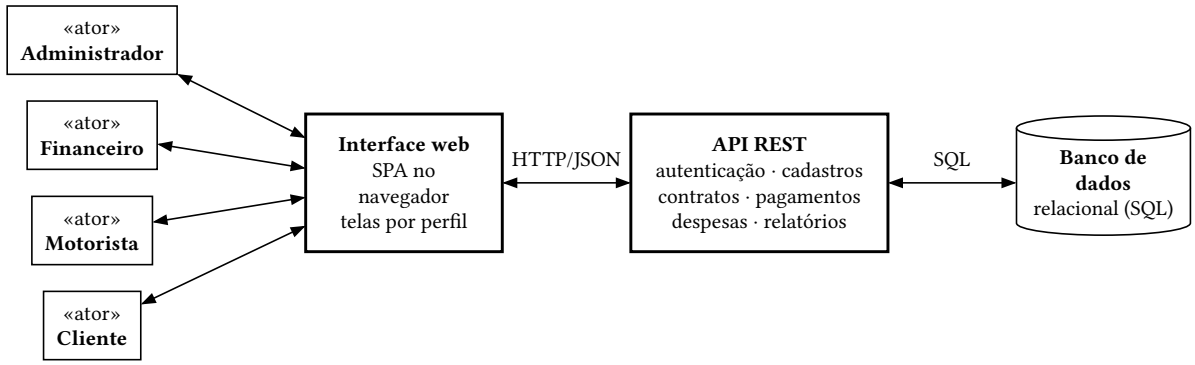
não abatem o repasse; (P4) autenticação por *token* com expiração e autorização por papel (RBAC); (P5) a situação "em atraso" é derivada da data de vencimento, não armazenada; (P6) a interface web é uma aplicação de página única (SPA), mantida no mesmo repositório da API, que acessa dados exclusivamente por ela; (P7) há um só *front-end*, cujos menus e telas se adaptam ao papel do usuário; (P8) o uso é em navegador (computador e celular), sem aplicativo nativo.

4 VISÃO GERAL E CASOS DE USO

4.1 CONTEXTO DO SISTEMA

O sistema é formado pela **interface web**, executada no navegador, e pela **API REST**, que persiste os dados em um banco relacional. A interface é o único cliente previsto da API e não acessa o banco diretamente; ferramentas de teste podem consumir a API do mesmo modo. Não há integrações externas.

Figura 4 – Diagrama de contexto.



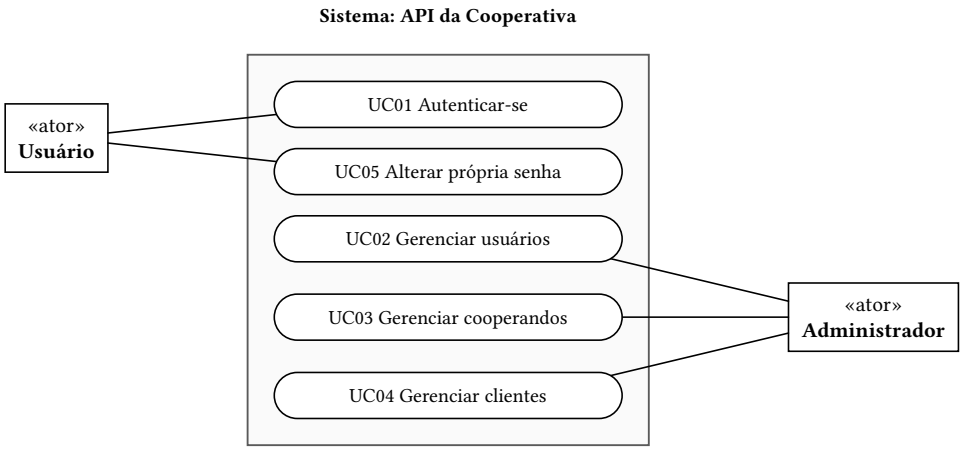
Fonte: Elaborado pelos autores.

4.2 CASOS DE USO

Os casos de uso estão agrupados em três módulos. Todos os perfis são especializações de *Usuário* autenticado (herdam UC01 e UC05). Todos os casos de uso são realizados por meio da interface web.

Módulo A — acesso e cadastros (inclui o CRUD de uma tabela: Usuário).

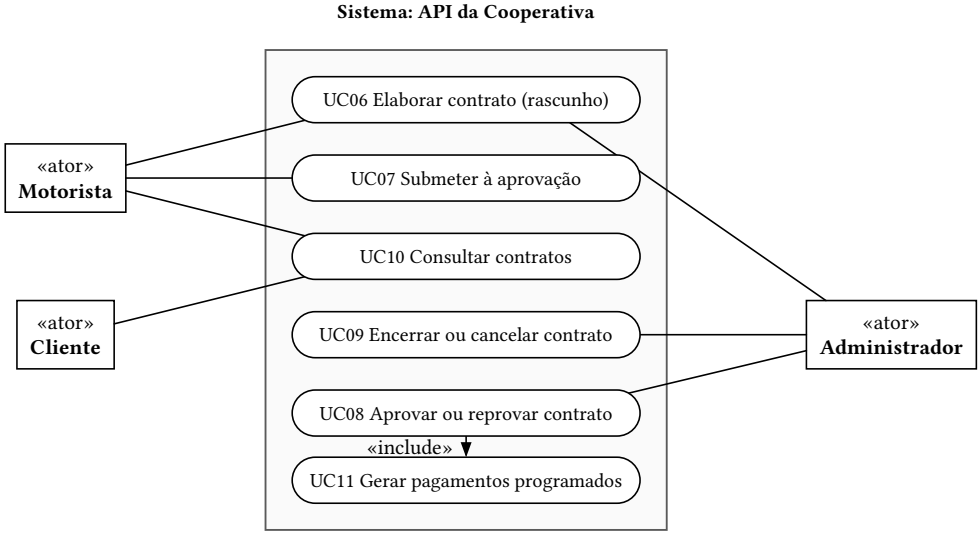
Figura 5 – Casos de uso do módulo de acesso e cadastros.



Fonte: Elaborado pelos autores.

Módulo B — contratos (CRUD relacionando Contrato, Cooperando, Cliente e Pagamento).

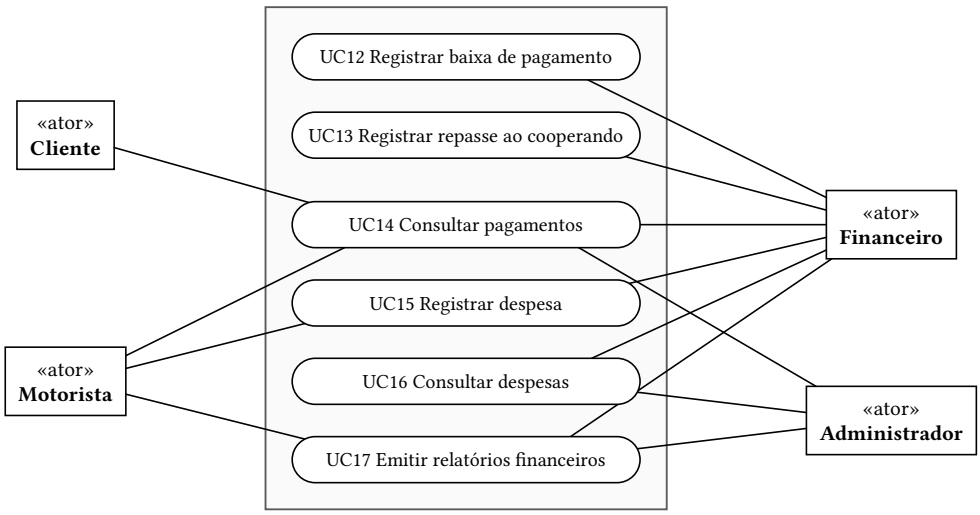
Figura 6 – Casos de uso do módulo de contratos. O administrador também atua em UC06 e UC07.



Fonte: Elaborado pelos autores.

Módulo C — financeiro (pagamentos, repasses, despesas e relatórios).

Figura 7 – Casos de uso do módulo financeiro. O motorista só enxerga e lança dados próprios (RN10).



Fonte: Elaborado pelos autores.

5 REGRAS DE NEGÓCIO

Quadro 6 – Regras de negócio.

ID	Regra
RN01	Todo contrato vincula exatamente um cliente e um cooperando. A cooperativa não é parte do contrato; seu papel é controlar o fluxo financeiro e coletar a taxa.
RN02	A taxa é definida por contrato: `taxa_tipo` ∈ {PERCENTUAL, FIXO}. PERCENTUAL exige `taxa_valor` maior que 0 e menor ou igual a 100; FIXO exige `taxa_valor` > 0 (valor em reais por pagamento). A taxa só pode ser alterada enquanto o contrato estiver em RASCUNHO.
RN03	`valor_taxa` = `valor_bruto` × `taxa_valor` ÷ 100 (percentual) ou `taxa_valor` (fixo), arredondado a 2 casas; `valor_liquido` = `valor_bruto` − `valor_taxa`. A taxa nunca pode exceder o valor bruto. Os três valores são gravados no pagamento na aprovação e congelados.
RN04	O cliente paga à cooperativa. Cada pagamento tem dois eventos manuais e sequenciais: *baixa* (recebimento do cliente) e *repasse* (transferência do líquido ao cooperando).
RN05	A modalidade do contrato é ÚNICO (1 pagamento, vencimento em `data_inicio`) ou MENSAL (N parcelas mensais, N ≥ 2, com `dia_vencimento` entre 1 e 28, a primeira na primeira ocorrência do dia em ou após `data_inicio`). Os pagamentos são gerados na aprovação, e a soma dos brutos é igual ao `valor_total` (resíduo de arredondamento na última parcela).
RN06	Só contratos ATIVOS aceitam baixa. O encerramento exige que nenhum pagamento esteja PREVISTO ou PAGO sem repasse. O cancelamento muda para CANCELADO os pagamentos PREVISTO; pagamentos já PAGO ou REPASSADO são preservados.
RN07	O repasse só é permitido sobre pagamento PAGO. Ao repassar, o valor transferido é o `valor_liquido` registrado.
RN08	Toda despesa tem `origem`: COOPERATIVA (sem cooperando e sem contrato) ou COOPERANDO (cooperando obrigatório; contrato opcional, e, se informado, deve pertencer ao cooperando). Valor > 0; categoria pertence a um conjunto fixo (combustível, pedágio, manutenção, administrativa, tributos, outros).
RN09	Despesas de cooperando são informativas: não reduzem o valor repassado.
RN10	Escopo de dados: o motorista acessa apenas seus contratos, pagamentos e despesas; o cliente acessa apenas seus contratos e pagamentos (leitura); o motorista também consulta os dados de identificação dos clientes para elaborar contratos; administrador e financeiro acessam todos os dados conforme Capítulo 11. Recurso fora do escopo responde 404.
RN11	Não há exclusão física de registros referenciados. Usuários, cooperandos e clientes são inativados; contratos só são excluídos em RASCUNHO; pagamentos nunca são excluídos.
RN12	Ciclo do contrato: RASCUNHO → PENDENTE_APROVACAO → ATIVO → ENCERRADO; CANCELADO a partir de qualquer estado não final. Reprovar devolve a RASCUNHO com `motivo_reprovacao`. Somente RASCUNHO é editável. Somente o administrador aprova, reprova, encerra ou cancela. Quem elabora o contrato não aprova o próprio.
RN13	Cooperando ou cliente inativo não pode ser vinculado a novo contrato.
RN14	Usuário de papel MOTORISTA está vinculado a exatamente um cooperando e o de papel CLIENTE a exatamente um cliente (relação 1:1); ADMINISTRADOR e FINANCEIRO não têm vínculo. E-mail é único.
RN15	Aprovação, baixa e repasse registram o usuário responsável e o instante da ação.

Fonte: Elaborado pelos autores.

6 REQUISITOS FUNCIONAIS

Prioridade: **E** = essencial, **I** = importante, **D** = desejável (escala em três níveis derivada de MoSCoW). Atores: ADM = administrador, FIN = financeiro, MOT = motorista, CLI = cliente.

6.1 AUTENTICAÇÃO E ACESSO

ID	Requisito	Prio.	Ator(es)	Regras
RF-AUT-01	Autenticar por e-mail e senha, emitindo *token* de acesso com expiração.	E	Todos	RN14
RF-AUT-02	Autorizar cada requisição conforme o papel e a propriedade do recurso (Capítulo 11).	E	Todos	RN10
RF-AUT-03	Permitir que o usuário altere a própria senha informando a senha atual.	I	Todos	—

6.2 USUÁRIOS — *CRUD DE UMA TABELA*

ID	Requisito	Prio.	Ator(es)	Regras
RF-USR-01	Criar usuário (nome, e-mail único, senha, papel).	E	ADM	RN14
RF-USR-02	Listar e consultar usuários, com filtro por papel e situação, paginados.	E	ADM	—
RF-USR-03	Atualizar dados e papel de um usuário.	E	ADM	RN14
RF-USR-04	Inativar e reativar usuário; remoção física apenas sem vínculos.	E	ADM	RN11

6.3 COOPERANDOS E CLIENTES

ID	Requisito	Prio.	Ator(es)	Regras
RF-COO-01	Cadastrar cooperando (matrícula, nome, CPF, telefone, data de ingresso) junto com seu usuário MOTORISTA, em uma única transação.	E	ADM	RN14
RF-COO-02	Listar e consultar cooperandos por nome, matrícula e situação.	E	ADM, FIN	—
RF-COO-03	Atualizar dados do cooperando.	E	ADM	—
RF-COO-04	Inativar cooperando, bloqueando novos contratos.	E	ADM	RN11, RN13
RF-CLI-01	Cadastrar cliente (nome, CPF/CNPJ, telefone, e-mail) junto com seu usuário CLIENTE, em uma única transação.	E	ADM	RN14
RF-CLI-02	Listar e consultar clientes por nome, documento e situação (ao motorista, apenas identificação).	E	ADM, FIN, MOT	—
RF-CLI-03	Atualizar dados do cliente.	E	ADM	—
RF-CLI-04	Inativar cliente, bloqueando novos contratos.	E	ADM	RN11, RN13

6.4 CONTRATOS — *CRUD RELACIONADO (CONTRATO, COOPERANDO, CLIENTE, PAGAMENTO)*

ID	Requisito	Prio.	Ator(es)	Regras
RF-CTR-01	Criar contrato em RASCUNHO informando cooperando, cliente, descrição do serviço, valor total, modalidade, parcelas, dia de vencimento, vigência e taxa (tipo e valor).	E	MOT, ADM	RN01–03, RN05, RN13
RF-CTR-02	Editar contrato em RASCUNHO.	E	MOT, ADM	RN02, RN12
RF-CTR-03	Submeter à aprovação (RASCUNHO → PENDENTE_APROVACAO), validando a completude dos dados.	E	MOT, ADM	RN12
RF-CTR-04	Aprovar contrato (→ ATIVO), gerar o cronograma de pagamentos na mesma transação e registrar o aprovador.	E	ADM	RN03, RN05, RN12, RN15
RF-CTR-05	Reprovar contrato com motivo (→ RASCUNHO).	E	ADM	RN12
RF-CTR-06	Encerrar contrato ATIVO (→ ENCERRADO).	E	ADM	RN06
RF-CTR-07	Cancelar contrato (→ CANCELADO), cancelando os pagamentos PREVISTO.	E	ADM	RN06
RF-CTR-08	Listar contratos com filtros (situação, cooperando, cliente, período), paginados e limitados ao escopo do papel.	E	Todos	RN10
RF-CTR-09	Detalhar contrato com dados do cooperando, do cliente e lista de pagamentos.	E	Todos	RN10
RF-CTR-10	Excluir contrato em RASCUNHO.	I	MOT, ADM	RN11

6.5 PAGAMENTOS E REPASSES

ID	Requisito	Prio.	Ator(es)	Regras
RF-PAG-01	Listar pagamentos com filtros (contrato, situação, vencimento, em atraso), limitados ao escopo do papel.	E	Todos	RN10
RF-PAG-02	Registrar baixa (PREVISTO → PAGO) com data de recebimento e observação.	E	FIN	RN04, RN06, RN15
RF-PAG-03	Registrar repasse ao cooperando (PAGO → REPASSADO) com data.	E	FIN	RN07, RN15
RF-PAG-04	Estornar baixa ainda não repassada (PAGO → PREVISTO), com justificativa.	D	ADM, FIN	RN15
RF-PAG-05	Consultar pagamentos em atraso (vencimento anterior a hoje e situação PREVISTO).	I	ADM, FIN	—

6.6 DESPESAS — *CRUD RELACIONADO (DESPESA, COOPERANDO, CONTRATO, USUÁRIO)*

ID	Requisito	Prio.	Ator(es)	Regras
RF-DSP-01	Registrar despesa (origem, categoria, descrição, valor, data; cooperando e contrato quando aplicável). Financeiro e administrador lançam as da cooperativa; o motorista lança as próprias.	E	FIN, ADM, MOT	RN08
RF-DSP-02	Listar despesas com filtros (origem, categoria, cooperando, período), limitadas ao escopo do papel.	E	ADM, FIN, MOT	RN10
RF-DSP-03	Atualizar despesa (o motorista, apenas as suas).	E	ADM, FIN, MOT	RN08, RN10
RF-DSP-04	Excluir despesa (o motorista, apenas as suas).	I	ADM, FIN, MOT	RN10
RF-DSP-05	Totalizar despesas por categoria e período.	I	ADM, FIN, MOT	RN10

6.7 RELATÓRIOS

ID	Requisito	Prio.	Ator(es)	Regras
RF-REL-01	Extrato do cooperando no período: pagamentos (bruto, taxa, líquido, situação do repasse) e despesas.	I	MOT, ADM, FIN	RN09, RN10
RF-REL-02	Resumo financeiro da cooperativa no período: taxas arrecadadas, valores a repassar e repassados, despesas da cooperativa e saldo.	I	ADM, FIN	RN03, RN09
RF-REL-03	Inadimplência: pagamentos em atraso agrupados por cliente e por cooperando.	D	ADM, FIN	—
RF-REL-04	Indicadores do painel inicial conforme o papel: contratos pendentes de aprovação, pagamentos a receber, em atraso e a repassar, contratos ativos e próximos vencimentos.	I	Todos	RN10

6.8 INTERFACE WEB

A interface é uma aplicação de página única que consome exclusivamente a API (P6) e reflete a matriz de permissões (Capítulo 11). Os requisitos abaixo descrevem funções e comportamentos, e não a aparência.

ID	Requisito	Prio.	Ator(es)	Regras
RF-WEB-01	Oferecer tela de *login* e opção de *logout*; manter a sessão enquanto o *token* for válido e redirecionar ao *login* quando a API responder 401.	E	Todos	RF-AUT-01
RF-WEB-02	Exibir menus, telas e botões apenas para as funções permitidas ao papel do usuário.	E	Todos	RN10, RN12
RF-WEB-03	Fornecer listagens de usuários, cooperandos, clientes, contratos, pagamentos e despesas com paginação, filtros e ordenação.	E	Conforme papel	RNF-08

ID	Requisito	Prio.	Ator(es)	Regras
RF-WEB-04	Fornecer formulários de cadastro e edição com validação prévia no navegador e exibição, junto ao campo, dos erros devolvidos pela API.	E	ADM, FIN, MOT	RN02–RN08, RNF-07
RF-WEB-05	Exibir o detalhe do contrato (cooperando, cliente e cronograma de pagamentos) com as ações de transição disponíveis para o status e o papel: submeter, aprovar, reprovar (com motivo), encerrar e cancelar.	E	Todos	RN12
RF-WEB-06	Permitir ao financeiro registrar baixa e repasse a partir da lista de pagamentos, destacando os pagamentos em atraso.	E	FIN	RN04, RN07
RF-WEB-07	Permitir lançar, editar e excluir despesas, com campos que se adaptam à origem (cooperativa ou cooperando).	E	ADM, FIN, MOT	RN08
RF-WEB-08	Apresentar painel inicial com os indicadores de RF--REL-04 próprios do papel.	I	Todos	RF-REL-04
RF-WEB-09	Apresentar telas de extrato do cooperando, resumo financeiro e inadimplência, com seleção de período.	I	MOT, ADM, FIN	RF-REL-01 a 03
RF-WEB-10	Pedir confirmação antes de ações irreversíveis (cancelar, excluir, estornar) e informar o resultado de cada operação.	E	Todos	—
RF-WEB-11	Apresentar ao cliente apenas consulta, sem controles de criação ou edição.	E	CLI	RN10

7 REQUISITOS NÃO FUNCIONAIS

Quadro 7 – Requisitos não funcionais, com categorias de qualidade conforme a ISO/IEC 25010 (ISO/IEC, 2011).

ID	Categoria	Requisito	Verificação
RNF-01	Segurança	Senhas armazenadas com *hash* adaptativo (bcrypt ou Argon2) e nunca retornadas pela API.	Inspeção e teste
RNF-02	Segurança	*Token* com expiração máxima de 60 min; HTTPS na implantação.	Teste
RNF-03	Segurança	401 para requisição não autenticada; 403 para papel sem permissão; 404 para recurso fora do escopo.	Teste de integração
RNF-04	Integridade	Valores monetários em `DECIMAL(12,2)` (nunca ponto flutuante); chaves estrangeiras, unicidade e `CHECK` no banco.	Inspeção do esquema
RNF-05	Integridade	Operações compostas (aprovação, cadastro de cooperando/cliente com usuário) em transação atômica.	Teste de integração
RNF-06	Padrão REST	Recursos como substantivos no plural; verbos HTTP semânticos; *status* 200, 201, 204, 400, 401, 403, 404, 409 e 422; JSON em UTF-8, conforme o estilo REST (FIELDING, 2000).	Revisão do OpenAPI
RNF-07	Erros	Corpo de erro padronizado conforme a RFC 9457 (NOTTINGHAM; WILDE; DALAL, 2023), com lista de campos inválidos.	Teste
RNF-08	Usabilidade da API	Listagens paginadas (`page`, `size`, máximo 100) e ordenáveis.	Teste
RNF-09	Documentação	Especificação OpenAPI 3.x atualizada a cada entrega, com exemplos.	Revisão
RNF-10	Desempenho	Meta proposta: listagens com p95 ≤ 500 ms para até 10 mil registros, em ambiente local.	Medição simples
RNF-11	Testabilidade	Meta proposta: toda regra RN coberta por teste automatizado; cobertura ≥ 70% da camada de serviço.	Relatório de cobertura
RNF-12	Manutenibilidade	Arquitetura em camadas (controlador, serviço, repositório); regras de negócio somente na camada de serviço; migrações versionadas do esquema.	Revisão de código
RNF-13	Auditoria	Todas as tabelas com `criado_em` e `atualizado_em`; baixa, repasse e aprovação gravam usuário e instante.	Inspeção
RNF-14	Interoperabilidade	Datas e instantes em ISO 8601 (instantes em UTC; vencimentos como data simples).	Teste
RNF-15	Portabilidade	Banco SQL relacional; execução reproduzível via README e variáveis de ambiente.	Execução limpa
RNF-16	Compatibilidade	Interface web funcional nas duas versões mais recentes de Chrome, Firefox, Edge e Safari.	Teste manual
RNF-17	Usabilidade	Interface utilizável em telas de 360 px de largura até computador (layout responsivo), com indicação de carregamento e de erro.	Teste manual
RNF-18	Acessibilidade	Meta proposta: formulários com rótulos associados, contraste adequado e uso completo por teclado (nível básico do WCAG 2.1 AA).	Verificação com ferramenta

ID	Categoria	Requisito	Verificação
RNF-19	Segurança da interface	*Token* guardado de forma que reduza exposição a XSS; saídas escapadas; CORS da API restrito à origem do *front-end*; HTTPS na implantação.	Revisão e teste
RNF-20	Desempenho percebido	Meta proposta: tela inicial utilizável em até 3 s em rede local; retorno visual a cada requisição.	Medição simples
RNF-21	Localização	Textos em português do Brasil; datas no formato dd/mm/aaaa e valores em reais na exibição (a API mantém ISO 8601 e decimais).	Inspeção

Fonte: Elaborado pelos autores.

8 RESTRIÇÕES E ATENDIMENTO AO ESCOPO ACADÊMICO

Quadro 8 – Mapeamento entre a restrição da disciplina e a especificação.

Exigência da disciplina	Como é atendida	Requisitos
CRUD de uma tabela	Módulo de Usuários, tabela independente (papel como enumeração).	RF-USR-01 a 04
CRUD relacionando ao menos 3 tabelas	Contratos: `contrato` + `cooperando` + `cliente` + `pagamento` (4 tabelas). Despesas: `despesa` + `cooperando` + `contrato` + `usuario` (4 tabelas).	RF-CTR-01 a 10; RF--DSP-01 a 05
Sem gestão de veículos	Nenhuma entidade de veículo; dados do serviço ficam na descrição do contrato.	Capítulo 5 (RN01)
Sem integrações externas	Baixa e repasse manuais.	RF-PAG-02, RF-PAG-03
Interface web	Aplicação de página única que consome a API e exibe as funções por papel.	RF-WEB-01 a 11; RNF-16 a 21

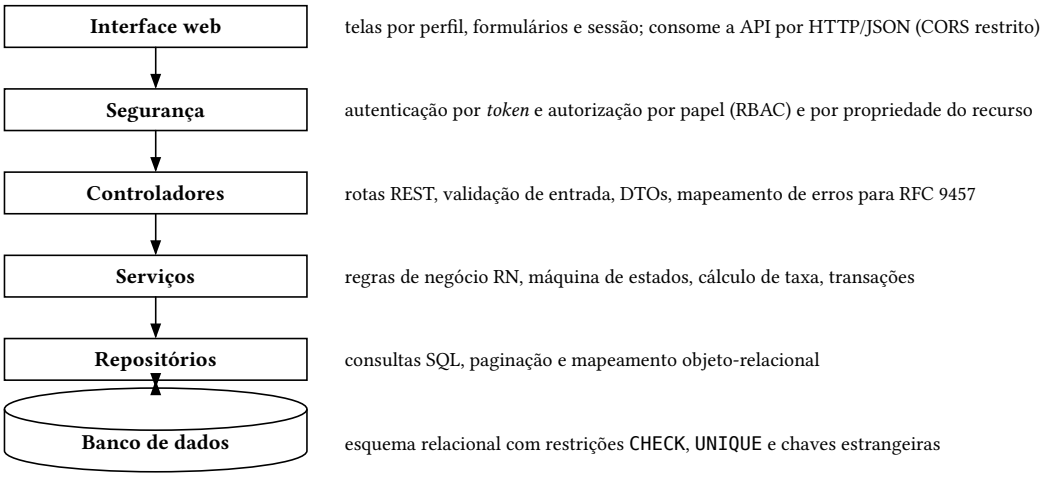
Fonte: Elaborado pelos autores.

9 MODELAGEM E PROJETO PRELIMINAR

9.1 ARQUITETURA EM CAMADAS

O sistema tem duas partes: a interface web e a API em camadas, ambas independentes de linguagem ou *framework*; a interface comunica-se com a API somente por HTTP/JSON. Na API, as regras de negócio (Capítulo 5) residem exclusivamente na camada de serviço, o que as torna testáveis sem HTTP nem banco.

Figura 8 – Arquitetura em camadas e fluxo de uma requisição. Aspectos transversais: auditoria (`criado_em`, usuário), *logs* e configuração por ambiente.

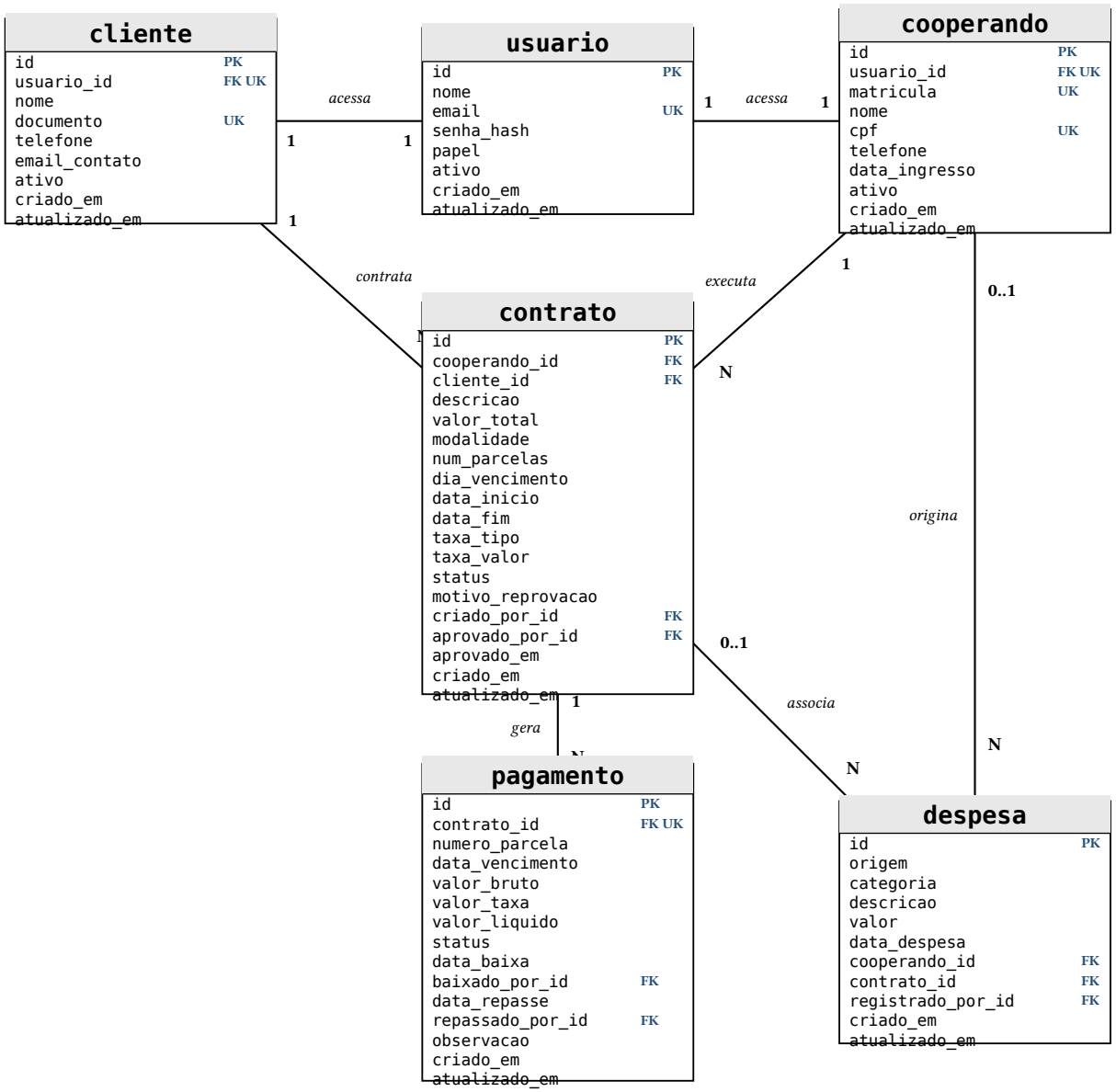


Fonte: Elaborado pelos autores.

9.2 MODELO DE DADOS

O modelo segue a abordagem entidade-relacionamento e o mapeamento relacional descritos em Elmasri e Navathe (2018). O esquema é definido uma única vez abaixo e reutilizado no diagrama entidade-relacionamento e no dicionário de dados. Colunas de auditoria que referenciam `usuario` (`criado_por_id`, `aprovado_por_id`, `baixado_por_id`, `repassado_por_id`, `registrado_por_id`) não são desenhadas no DER para preservar a legibilidade.

Figura 9 – Diagrama entidade-relacionamento (PK = chave primária, FK = estrangeira, UK = única). Veículos não são modelados.



Fonte: Elaborado pelos autores.

Quadro 9 – Dicionário de dados: entidade `usuario`

Coluna	Tipo	Restrições
`id`	BIGINT	PK, auto-incremento
`nome`	VARCHAR(120)	NN
`email`	VARCHAR(160)	UK, NN
`senha_hash`	VARCHAR(255)	NN
`papel`	ENUM	NN: ADMINISTRADOR, FINANCEIRO, MOTORISTA, CLIENTE
`ativo`	BOOLEAN	NN, padrão verdadeiro
`criado_em`	TIMESTAMP	NN
`atualizado_em`	TIMESTAMP	NN

Fonte: Elaborado pelos autores.

Quadro 10 – Dicionário de dados: entidade `cooperando`

Coluna	Tipo	Restrições
`id`	BIGINT	PK, auto-incremento

Coluna	Tipo	Restrições
usuario_id	BIGINT	FK → usuario, UK, NN (relação 1:1)
matricula	VARCHAR(20)	UK, NN
nome	VARCHAR(120)	NN
cpf	CHAR(11)	UK, NN
telefone	VARCHAR(20)	—
data_ingresso	DATE	NN
ativo	BOOLEAN	NN, padrão verdadeiro
criado_em	TIMESTAMP	NN
atualizado_em	TIMESTAMP	NN

Fonte: Elaborado pelos autores.

Quadro 11 – Dicionário de dados: entidade `cliente`

Coluna	Tipo	Restrições
id	BIGINT	PK, auto-incremento
usuario_id	BIGINT	FK → usuario, UK, NN (relação 1:1)
nome	VARCHAR(120)	NN
documento	VARCHAR(14)	UK, NN (CPF ou CNPJ, só dígitos)
telefone	VARCHAR(20)	—
email_contato	VARCHAR(160)	—
ativo	BOOLEAN	NN, padrão verdadeiro
criado_em	TIMESTAMP	NN
atualizado_em	TIMESTAMP	NN

Fonte: Elaborado pelos autores.

Quadro 12 – Dicionário de dados: entidade `contrato`

Coluna	Tipo	Restrições
id	BIGINT	PK, auto-incremento
cooperando_id	BIGINT	FK → cooperando, NN
cliente_id	BIGINT	FK → cliente, NN
descricao	VARCHAR(500)	NN
valor_total	DECIMAL(12,2)	NN, CHECK > 0
modalidade	ENUM	NN: UNICO, MENSAL
num_parcelas	SMALLINT	NN, CHECK ≥ 1 (1 se UNICO)
dia_vencimento	SMALLINT	CHECK entre 1 e 28; obrigatório se MENSAL
data_inicio	DATE	NN
data_fim	DATE	CHECK ≥ data_inicio
taxa_tipo	ENUM	NN: PERCENTUAL, FIXO
taxa_valor	DECIMAL(12,2)	NN, CHECK > 0 (≤ 100 se PERCENTUAL)
status	ENUM	NN: RASCUNHO, PENDENTE_APROVACAO, ATIVO, ENCERRADO, CANCELADO
motivo_reprovacao	VARCHAR(300)	—
criado_por_id	BIGINT	FK → usuario, NN
aprovado_por_id	BIGINT	FK → usuario
aprovado_em	TIMESTAMP	—
criado_em	TIMESTAMP	NN
atualizado_em	TIMESTAMP	NN

Fonte: Elaborado pelos autores.

Quadro 13 – Dicionário de dados: entidade `pagamento`

Coluna	Tipo	Restrições
id	BIGINT	PK, auto-incremento

Coluna	Tipo	Restrições
contrato_id	BIGINT	FK ⟶ contrato, NN; UK com numero_parcela
numero_parcela	SMALLINT	NN, CHECK ≥ 1
data_vencimento	DATE	NN
valor_bruto	DECIMAL(12,2)	NN, CHECK > 0
valor_taxa	DECIMAL(12,2)	NN, CHECK ≥ 0 e ≤ valor_bruto
valor_liquido	DECIMAL(12,2)	NN, = valor_bruto − valor_taxa
status	ENUM	NN: PREVISTO, PAGO, REPASSADO, CANCELADO
data_baixa	DATE	obrigatório se PAGO ou REPASSADO
baixado_por_id	BIGINT	FK ⟶ usuario
data_repasse	DATE	obrigatório se REPASSADO
repassado_por_id	BIGINT	FK ⟶ usuario
observacao	VARCHAR(300)	—
criado_em	TIMESTAMP	NN
atualizado_em	TIMESTAMP	NN

Fonte: Elaborado pelos autores.

Quadro 14 – Dicionário de dados: entidade `despesa`

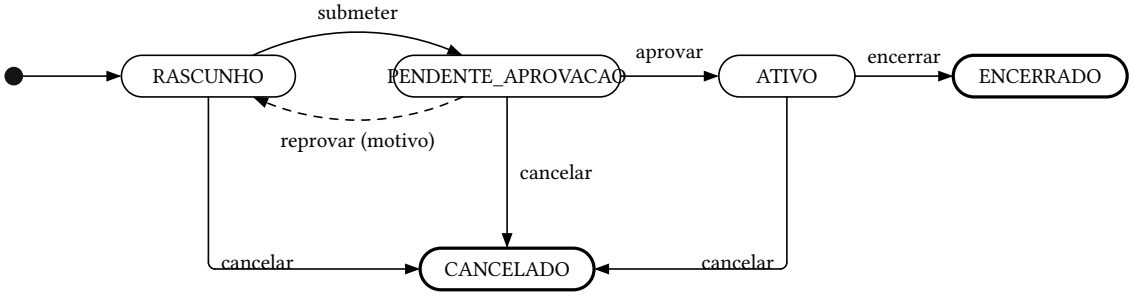
Coluna	Tipo	Restrições
id	BIGINT	PK, auto-incremento
origem	ENUM	NN: COOPERATIVA, COOPERANDO
categoria	ENUM	NN: COMBUSTIVEL, PEDAGIO, MANUTENCAO, ADMINISTRATIVA, TRIBUTOS, OUTROS
descricao	VARCHAR(300)	NN
valor	DECIMAL(12,2)	NN, CHECK > 0
data_despesa	DATE	NN
cooperando_id	BIGINT	FK ⟶ cooperando; NN se origem = COOPERANDO, nulo se COOPERATIVA
contrato_id	BIGINT	FK ⟶ contrato; nulo se COOPERATIVA; do mesmo cooperando se informado
registrado_por_id	BIGINT	FK ⟶ usuario, NN
criado_em	TIMESTAMP	NN
atualizado_em	TIMESTAMP	NN

Fonte: Elaborado pelos autores.

Restrições entre colunas (implementadas como `CHECK` ou verificadas na camada de serviço): `despesa.origem = 'COOPERATIVA'` implica `cooperando_id` e `contrato_id` nulos; `origem = 'COOPERANDO'` implica `cooperando_id` não nulo; a soma de `pagamento.valor_bruto` por contrato é igual a `contrato.valor_total` (RN05).

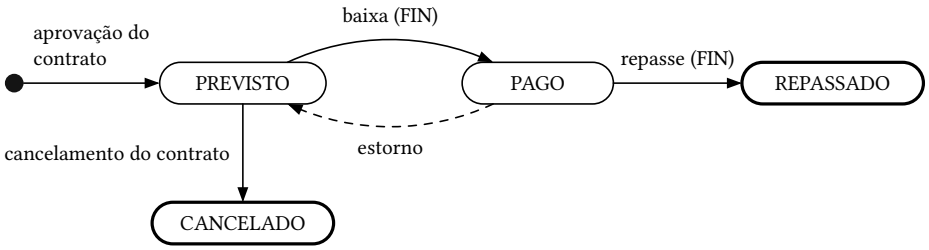
9.3 ESTADOS DO CONTRATO E DO PAGAMENTO

Figura 10 – Máquina de estados do contrato (RN12). Aprovar gera os pagamentos na mesma transação. ENCERRADO e CANCELADO são finais.



Fonte: Elaborado pelos autores.

Figura 11 – Máquina de estados do pagamento (RN04, RN06, RN07). *Em atraso* é derivado: situação PREVISTO com vencimento anterior à data atual (P5).

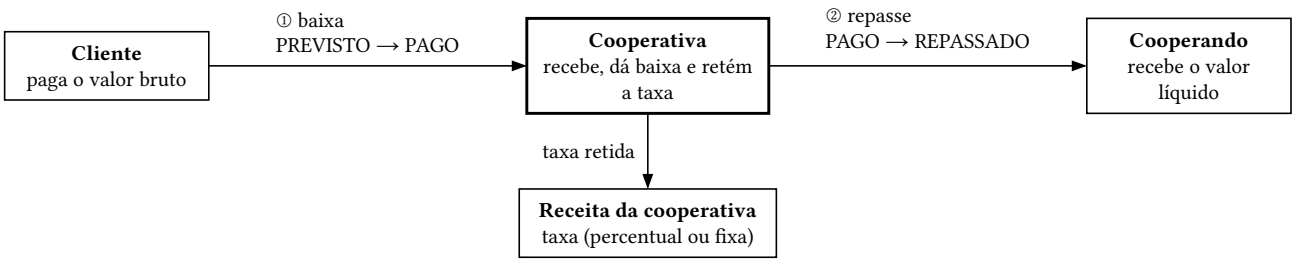


Fonte: Elaborado pelos autores.

9.4 FLUXO FINANCEIRO

Figura 12 – Fluxo do dinheiro: o contrato é entre cliente e cooperando; a cooperativa intermedeia.

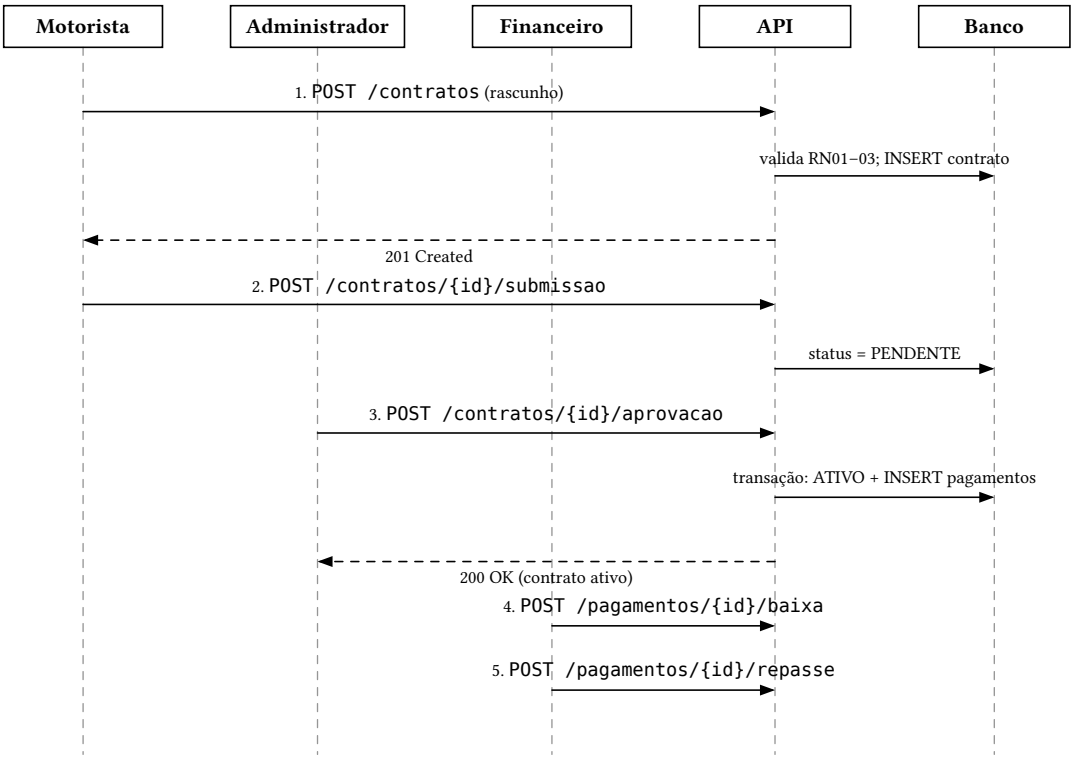
`valor_liquido = valor_bruto − valor_taxa` (RN03)



Fonte: Elaborado pelos autores.

9.5 SEQUÊNCIA DO CICLO DE VIDA DE UM CONTRATO

Figura 13 – Sequência: da elaboração do contrato ao repasse (as chamadas partem da interface web e cada passo valida papel, escopo e estado).

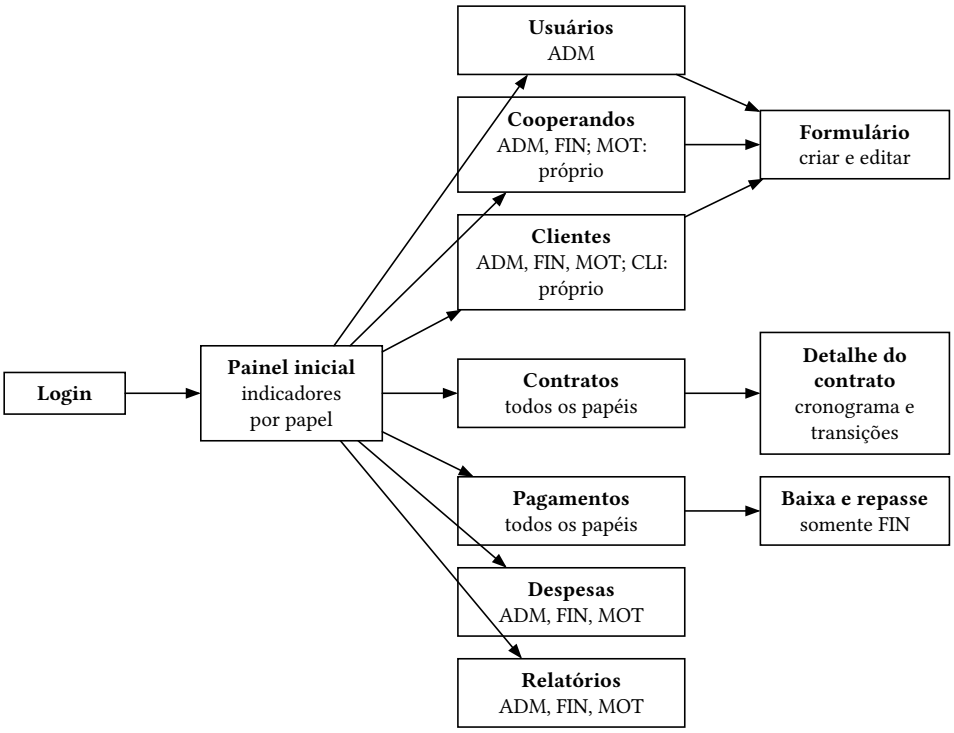


Fonte: Elaborado pelos autores.

9.6 MAPA DE NAVEGAÇÃO DA INTERFACE WEB

O mapa mostra a estrutura de telas e o acesso por papel. Telas de listagem seguem o mesmo padrão (RF-WEB-03), e as de cadastro usam um formulário comum de criação e edição (RF--WEB-04).

Figura 14 – Mapa de navegação por papel (estrutura de telas, sem projeto visual). As despesas e os contratos em rascunho também usam o formulário.



Fonte: Elaborado pelos autores.

10 CONTRATO DA API (RECURSOS E OPERAÇÕES)

10.1 CONVENÇÕES

Base `/api/v1`; representação JSON (UTF-8); identificadores numéricos; datas ISO 8601. Operações de transição de estado (submeter, aprovar, baixar etc.) são modeladas como *comandos* por `POST` em sub-recurso, pois não são simples atualizações de campo e carregam regras próprias. Códigos de resposta: `200` (consulta/atualização), `201` (criação, com `Location`), `204` (exclusão), `400` (requisição malformada), `401`, `403`, `404`, `409` (transição inválida ou conflito de estado), `422` (regra de negócio violada). Listagens aceitam `page`, `size`, `sort` e filtros por *query string*. Como a interface web é servida em origem própria, a API habilita CORS somente para a origem configurada do *front-end*.

Quadro 15 – Recursos da API. O prefixo `RF-` foi omitido na última coluna.

Método	Rota	Descrição	Papéis	RF
`POST`	`/auth/login`	Autentica e devolve o *token*.	Público	`AUT-01`
`POST`	`/auth/alterar-senha`	Altera a própria senha.	Todos	`AUT-03`
`POST`	`/usuarios`	Cria usuário.	ADM	`USR-01`
`GET`	`/usuarios`	Lista usuários (filtros: papel, ativo).	ADM	`USR-02`
`GET`	`/usuarios/{id}`	Consulta usuário.	ADM	`USR-02`
`PUT`	`/usuarios/{id}`	Atualiza usuário.	ADM	`USR-03`
`PATCH`	`/usuarios/{id}/situacao`	Inativa ou reativa usuário.	ADM	`USR-04`
`POST`	`/cooperandos`	Cadastra cooperando e usuário (transação).	ADM	`COO-01`
`GET`	`/cooperandos`	Lista cooperandos.	ADM, FIN	`COO-02`
`GET`	`/cooperandos/{id}`	Consulta cooperando (o motorista, apenas o próprio).	ADM, FIN, MOT	`COO-02`
`PUT`	`/cooperandos/{id}`	Atualiza cooperando.	ADM	`COO-03`
`PATCH`	`/cooperandos/{id}/situacao`	Inativa ou reativa cooperando.	ADM	`COO-04`
`POST`	`/clientes`	Cadastra cliente e usuário (transação).	ADM	`CLI-01`
`GET`	`/clientes`	Lista clientes.	ADM, FIN, MOT	`CLI-02`
`GET`	`/clientes/{id}`	Consulta cliente (o cliente, apenas o próprio).	ADM, FIN, MOT, CLI	`CLI-02`
`PUT`	`/clientes/{id}`	Atualiza cliente.	ADM	`CLI-03`
`PATCH`	`/clientes/{id}/situacao`	Inativa ou reativa cliente.	ADM	`CLI-04`
`POST`	`/contratos`	Cria contrato em RASCUNHO.	MOT, ADM	`CTR-01`
`GET`	`/contratos`	Lista contratos (filtros: status, cooperando, cliente, período).	Todos	`CTR-08`
`GET`	`/contratos/{id}`	Detalha contrato com cooperando, cliente e pagamentos.	Todos	`CTR-09`
`PUT`	`/contratos/{id}`	Edita contrato em RASCUNHO.	MOT, ADM	`CTR-02`
`DELETE`	`/contratos/{id}`	Exclui contrato em RASCUNHO.	MOT, ADM	`CTR-10`
`POST`	`/contratos/{id}/submissao`	RASCUNHO → PENDENTE_APROVACAO.	MOT, ADM	`CTR-03`
`POST`	`/contratos/{id}/aprovacao`	PENDENTE → ATIVO; gera pagamentos.	ADM	`CTR-04`

Método	Rota	Descrição	Papéis	RF
POST	/contratos/{id}/reprovacao	PENDENTE → RASCUNHO (corpo: motivo).	ADM	CTR-05
POST	/contratos/{id}/encerramento	ATIVO → ENCERRADO.	ADM	CTR-06
POST	/contratos/{id}/cancelamento	Estado não final → CANCELADO.	ADM	CTR-07
GET	/contratos/{id}/pagamentos	Lista pagamentos do contrato.	Todos	PAG-01
GET	/pagamentos	Lista pagamentos (filtros: situação, vencimento, em_atraso).	Todos	PAG-01
GET	/pagamentos/{id}	Consulta pagamento.	Todos	PAG-01
POST	/pagamentos/{id}/baixa	PREVISTO → PAGO.	FIN	PAG-02
POST	/pagamentos/{id}/repasse	PAGO → REPASSADO.	FIN	PAG-03
POST	/pagamentos/{id}/estorno	PAGO → PREVISTO (corpo: justificativa).	ADM, FIN	PAG-04
POST	/despesas	Registra despesa.	ADM, FIN, MOT	DSP-01
GET	/despesas	Lista despesas (filtros: origem, categoria, cooperando, período).	ADM, FIN, MOT	DSP-02
GET	/despesas/{id}	Consulta despesa.	ADM, FIN, MOT	DSP-02
PUT	/despesas/{id}	Atualiza despesa.	ADM, FIN, MOT	DSP-03
DELETE	/despesas/{id}	Exclui despesa.	ADM, FIN, MOT	DSP-04
GET	/despesas/totais	Totais por categoria e período.	ADM, FIN, MOT	DSP-05
GET	/cooperandos/{id}/extrato	Extrato do cooperando no período.	ADM, FIN, MOT	REL-01
GET	/relatorios/resumo-financeiro	Resumo financeiro da cooperativa.	ADM, FIN	REL-02
GET	/relatorios/inadimplencia	Pagamentos em atraso agrupados.	ADM, FIN	REL-03
GET	/painel/indicadores	Indicadores do painel conforme o papel.	Todos	REL-04

Fonte: Elaborado pelos autores.

11 MATRIZ DE PERMISSÕES

T = todos os registros; **P** = somente registros próprios (RN10); — = sem acesso. A interface web oculta os controles não permitidos (RF-WEB-02), mas a API é a autoridade final de autorização (RNF-03).

Quadro 16 – Autorização por papel (RBAC) com restrição por propriedade do recurso.

Recurso / operação	ADM	FIN	MOT	CLI
Usuários: criar, listar, editar, inativar	T	—	—	—
Alterar a própria senha	P	P	P	P
Cooperandos: criar, editar, inativar	T	—	—	—
Cooperandos: consultar	T	T	P	—
Clientes: criar, editar, inativar	T	—	—	—
Clientes: consultar	T	T	T (identificação)	P
Contratos: criar, editar e excluir rascunho; submeter	T	—	P	—
Contratos: aprovar, reprovar, encerrar, cancelar	T	—	—	—
Contratos: consultar	T	T	P	P
Pagamentos: consultar	T	T	P	P
Pagamentos: baixa e repasse	—	T	—	—
Pagamentos: estorno de baixa	T	T	—	—
Despesas: registrar, editar, excluir	T	T	P (origem COOPERANDO)	—
Despesas: consultar e totalizar	T	T	P	—
Relatório: extrato do cooperando	T	T	P	—
Relatórios: resumo financeiro e inadimplência	T	T	—	—

Fonte: Elaborado pelos autores.

12 CRITÉRIOS DE ACEITAÇÃO

Cenários mínimos, no formato *dado–quando–então*, que servirão de base aos testes de integração. Os demais requisitos são verificados conforme as colunas "Verificação" e as regras de negócio associadas.

Quadro 17 – Cenários de aceitação.

ID	Dado	Quando	Então
CA-01	Contrato PENDENTE, MENSAL, valor total 6000,00, 3 parcelas, taxa PERCENTUAL de 10.	O administrador aprova.	Status ATIVO; 3 pagamentos PREVISTO com bruto 2000,00, taxa 200,00 e líquido 1800,00 (RF-CTR-04; RN03, RN05).
CA-02	Contrato em RASCUNHO com taxa FIXA de 500,00 e parcela de 400,00.	O motorista submete.	`422` com o campo `taxa_valor` indicado (RN03).
CA-03	Pagamento PREVISTO.	O financeiro tenta registrar repasse.	`409`; situação permanece PREVISTO (RN07).
CA-04	Contrato do motorista B.	O motorista A consulta esse contrato.	`404` (RN10, RNF-03).
CA-05	Contrato PENDENTE.	O motorista tenta aprovar.	`403`; contrato permanece PENDENTE (RN12).
CA-06	Despesa de origem COOPERATIVA.	Informa-se `cooperando_id`.	`422` (RN08).
CA-07	Contrato ATIVO com pagamento PAGO e sem repasse.	O administrador tenta encerrar.	`409` (RN06).
CA-08	Motorista autenticado na interface.	Abre o detalhe de um contrato PENDENTE.	Nenhuma ação de aprovar ou reprovar é exibida (RF-WEB-02; RN12).
CA-09	Formulário de contrato com taxa fixa maior que a parcela.	O usuário envia e a API responde `422`.	A mensagem aparece junto ao campo `taxa_valor` (RF-WEB-04).
CA-10	Sessão com *token* expirado.	O usuário aciona qualquer tela e a API responde `401`.	A interface redireciona ao *login* (RF--WEB-01).

Fonte: Elaborado pelos autores.

13 RISCOS E GESTÃO DE MUDANÇAS

Quadro 18 – Registro de riscos.

Risco	Descrição	Mitigação
R1	Premissas P1–P5 não confirmadas pelo stakeholder geram retrabalho.	Validar as premissas no marco da fase 1 e registrar a resposta neste documento.
R2	Crescimento de escopo (veículos, pagamento online, front-end).	Escopo fechado na seção 1.3; mudanças somente por solicitação registrada e nova versão do documento.
R3	Ambiguidade da taxa fixa em contratos mensais.	Regra explícita em RN02 (por pagamento) e cenário de teste CA-02.
R4	Erros de arredondamento monetário.	`DECIMAL`, resíduo na última parcela (RN05) e testes de soma.
R5	Tempo insuficiente para todos os requisitos.	Priorização E/I/D; entrega incremental por módulo com testes junto de cada *feature*.
R6	Conflitos de integração no Git.	Ramos curtos, *pull requests* pequenos e revisão cruzada.
R7	A interface web amplia o esforço e ameaça o prazo.	Telas padronizadas e reutilizáveis (listagem, formulário, detalhe); entrega vertical por módulo; priorização E/I/D.

Fonte: Elaborado pelos autores.

Gestão de mudanças. Qualquer alteração de requisito após a aprovação do documento segue o fluxo: solicitação registrada → análise de impacto (RF, RN, DER, API, testes) → aprovação → atualização desta especificação em ramo `docs/*` e incremento da versão.

REFERÊNCIAS

CONVENTIONAL COMMITS. **Conventional Commits 1.0.0**. [S. l.]: [s. n.], [s. d.]. Disponível em: https://www.conventionalcommits.org/. Acesso em: 31 out. 2026.

DRIESSEN, Vincent. **A successful Git branching model**. [S. l.]: [s. n.], 2010. Disponível em: https://nvie.com/posts/a-successful-git-branching-model/. Acesso em: 31 out. 2026.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Informação e Ciência da Computação) – University of California, Irvine, 2000.

ISO/IEC. **ISO/IEC 25010:2011**: systems and software engineering – systems and software Quality Requirements and Evaluation (SQuaRE) – system and software quality models. Geneva: ISO, 2011.

ISO/IEC/IEEE. **ISO/IEC/IEEE 29148:2018**: systems and software engineering – life cycle processes – requirements engineering. Geneva: ISO, 2018.

NOTTINGHAM, Mark; WILDE, Erik; DALAL, Sanjay. **RFC 9457**: problem details for HTTP APIs. [S. l.]: Internet Engineering Task Force, 2023. Disponível em: https://www.rfc-editor.org/rfc/rfc9457. Acesso em: 31 out. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

PRESTON-WERNER, Tom. **Semantic Versioning 2.0.0**. [S. l.]: [s. n.], 2013. Disponível em: https://semver.org/. Acesso em: 31 out. 2026.

ROYCE, Winston W. Managing the development of large software systems. In: IEEE WESCON, 1970. **Proceedings** [...]. [S. l.]: IEEE, 1970.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2018.

WIEGERS, Karl; BEATTY, Joy. **Software requirements**. 3. ed. Redmond: Microsoft Press, 2013.